STRATHHACK 2026 · TEAM 22

Space Salvage

Code study guide: everything you need to understand the game well enough to present it and answer the judges' questions.

Your 2-hour study plan

Time	Read	Goal
0:00 – 0:10	1. The pitch & the big picture	Say what the game is and why it fits "Discord Games" in 60 seconds.
0:10 – 0:25	2. Godot crash course	Know the 10 Godot words the rest of the guide uses.
0:25 – 0:55	3. How the multiplayer works	Explain hosting, join codes, authority and RPCs. Judges ask about this most.
0:55 – 1:35	4. File-by-file tour	Know what every file does and its 2–3 key functions.
1:35 – 1:50	5. Mechanics deep dives	Tethers, relics, oxygen, radar, darkness, background, effects.
1:50 – 2:00	6. Judge Q&A drill + 7. cheat sheet	Answer every question out loud once.

Contents

1. The pitch & the big picture
2. Godot crash course
3. How the multiplayer works
4. File-by-file tour
5. Mechanics deep dives
6. Judge Q&A drill
7. Cheat sheet (numbers, controls, credits)

Tip: open the project next to this guide and look at each file as you read about it. Every function name in this guide exists in the code.

1 The pitch & the big picture

The game in one sentence

Space Salvage is a co-op multiplayer salvage game (think *Dave the Diver* in space, seen side-on like *Terraria*): your crew drifts out from a space station in zero gravity, grabs gold that's worth more the further out it is, and must get home before their air runs out, all while roped together and half-blind in the dark.

The core loop

1. **Dive:** leave the station through an airlock into zero-g. Oxygen starts draining.
2. **Salvage:** touch loot to grab it (scrap → gold bars → relics, worth more further out).
3. **Return & sell:** get back inside (air refills) and press **E** to sell. Money is shared by the whole team.
4. **Upgrade:** buy bigger oxygen tanks, faster jetpacks and longer tethers (keys 1–4).
5. **Win:** buy all 5 escape-pod repair parts → the pod launches → "YOU MADE IT HOME!"

Why it fits the theme ("Discord Games": communication is the mechanic)

The brief asked for a small multiplayer game where **talking is part of the gameplay**. We built three systems that make you *have* to talk:

Mechanic	Why you must communicate
Darkness + radar	Space is dark: divers only see what's in their helmet lamp. One player stays in the station (press **R**) and gets a radar showing all loot and everyone's oxygen. They have to guide the divers by voice: "*Left! LEFT! You've got 10% air!*"
Tethers	Every diver is always roped to a teammate. Stretch the rope too far and it snaps and you **both** black out. Pairs must agree where to go.
Heavy relics	$500 relics only move while **two** players hold on, and they drift to the midpoint between them. You have to coordinate direction.

> **Say this to the judges (60-second pitch):** "Space Salvage is a co-op game where your crew salvages gold in deep space, but nobody can do it alone. Divers are blind in the dark and roped together, so whoever stays on the station radar has to talk them to the loot and back before their air runs out. Heavy relics need two people pulling the same way, and if you drift apart your rope snaps and you both black out. It's chaotic, it's funny, and the whole game is about communicating, which is exactly the Discord Games theme."

The big picture: how the code is organised

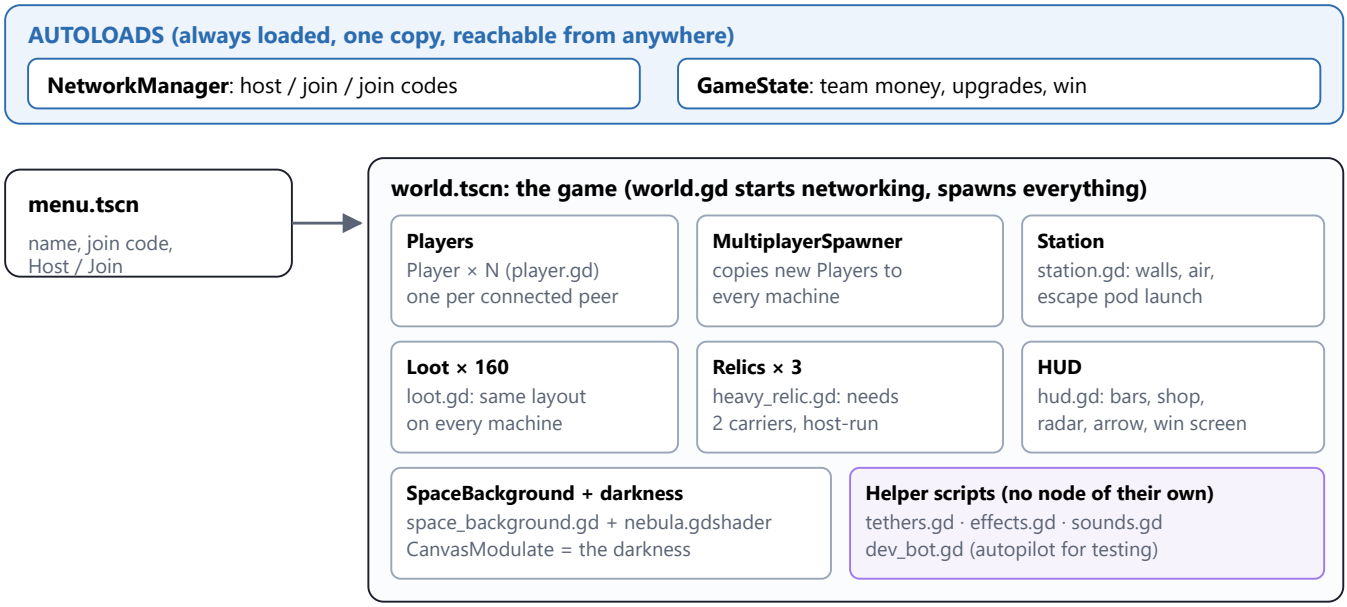


Figure 1: the menu loads the world scene; the world builds everything else. Two autoloads hold the networking and the shared team state.

The code is about 2,600 lines of GDScript across 15 scripts, plus one shader. Almost everything (UI, shapes, physics shapes) is built **in code** rather than in the editor, so scene files stay tiny and merge cleanly in git.

2 Godot crash course

You only need these ideas to follow the code. Godot is a free game engine; we use version 4.7 and its scripting language **GDScript** (looks like Python).

Word	What it means	Where we use it
Node	Every object in the game: a sprite, a sound, a camera, a timer. Nodes form a **tree** (parents and children).	Everything.
Scene (`.tscn`)	A saved tree of nodes you can load or copy ("instantiate"). Like a prefab.	`menu.tscn`, `world.tscn`, `player.tscn`, `loot.tscn`
Script (`.gd`)	Code attached to a node, giving it behaviour. `extends CharacterBody2D` says what kind of node it is. `class_name Player` gives it a global name.	One script per main object.
Lifecycle functions	`_ready()` runs once when the node appears. `_process(delta)` runs every frame (visuals). `_physics_process(delta)` runs 60× a second (movement). `delta` = seconds since the last frame, so speeds are "per second".	All scripts.
Signal	An event a node announces; other code `connect`s a function to it. E.g. `body_entered` when something touches an area.	Station detects players; GameState announces `game_won`.
Autoload	A script Godot loads once at startup and keeps forever, reachable by name from anywhere (a "singleton").	`NetworkManager`, `GameState`
Area2D / CharacterBody2D	Area2D detects overlaps (no collision). CharacterBody2D is a body you move with `move_and_slide()`.	Loot & station = Area2D. Player = CharacterBody2D.
`_draw()`	Custom drawing: `draw_rect`, `draw_circle`, `draw_line` … Call `queue_redraw()` to redraw.	Station, loot, relics, ropes, radar.
CanvasLayer	A separate drawing layer that ignores the camera. Used for UI and backgrounds.	HUD (in front), space background (behind).
Group	A tag on nodes, e.g. `add_to_group("loot")`. Any script can then find them all with `get_tree().get_nodes_in_group("loot")`.	The radar and bots find players, loot and relics this way.
Tween	Smoothly animates a value over time.	Blackout fade, pod launch, win zoom.
@rpc	"Remote procedure call": a function that can be called **on other players' machines** over the network. Explained in section 3.	Pickups, buying, effects.

Reading GDScript in 30 seconds

```
var speed := 300.0            # variable; := means "infer the type"
const MAX := 5                  # constant (UPPER_CASE = tuning knob)
func sell() -> void:            # function; -> void means it returns nothing
    if carried_value <= 0:
        return
for loot in get_tree().get_nodes_in_group("loot"):   # loop over every loot node
$Camera2D                        # shorthand for get_node("Camera2D"), a child node
"%s sold $%d" % [name, money]    # text formatting, like printf
```

3 How the multiplayer works

This is the part judges are most likely to probe. Learn these five ideas.

Idea 1: one player hosts, everyone else joins (ENet)

The host's game acts as the **server**. Others connect to it over the local network on **UDP port 9999** (up to 8 players), using ENet, a fast networking library built into Godot.

```
# network_manager.gd
func host() -> Error:
    var peer := ENetMultiplayerPeer.new()
    var err := peer.create_server(PORT, MAX_PLAYERS)   # 9999, 8
    ...
    multiplayer.multiplayer_peer = peer                # Godot now routes all networking through it

func join(ip: String) -> Error:
    var peer := ENetMultiplayerPeer.new()
    var err := peer.create_client(ip, PORT)
```

Every machine gets a **peer id**. The host is always **1**; clients get random big numbers. Each player node is **named after its peer id** (e.g. node `"1"`, node `"170686491"`).

Idea 2: join codes (3 numbers instead of an IP)

Laptops on the same Wi-Fi share the start of their IP address, so the host only shares the **end**. The host screen shows `Join code: 144`; the joiner types `144` and their game fills in the rest from **its own** IP. If the networks differ in the third number (big uni Wi-Fi), the backup code `117.144` works too.

```
func code_to_ip(code: String) -> String:
    var code_parts := code.split(".")
    ...
    var parts := ips[0].split(".")       # our own IP, e.g. ["10", "12", "117", "50"]
    parts.resize(4 - code_parts.size())  # keep the front, e.g. ["10", "12", "117"]
    parts.append_array(code_parts)       # add the code   -> ["10", "12", "117", "144"]
    return ".".join(parts)               # "10.12.117.144"
```

`local_ips()` ranks network adapters so **Wi-Fi/Ethernet come first** and virtual ones (VirtualBox, Bluetooth) last. Before that fix, a laptop could show the code of an adapter nobody could reach. If a join fails, the game gives up after 10 s with a helpful message instead of hanging.

Idea 3: authority (who is in charge of what)

Every node has one machine that is its **authority**: the only one allowed to change it. Everyone else just receives copies.

Thing	Authority	Why
Your player (movement, oxygen, cargo, rope physics)	**You** (your own machine)	Instant controls with no lag; friends trust each other.
Team money, upgrades, repair parts (`GameState`)	**Host**	So two people buying at once can't spend the same money.
Who picked up which loot	**Host**	First come, first served, with no duplicates.
Heavy relics (position, who's holding)	**Host**	It's shared by two players, so one machine must decide.

```
# player.gd: the player node is named after its owner, so it hands itself to them
func _enter_tree() -> void:
    set_multiplayer_authority(name.to_int())
    _setup_replication()
```

Idea 4: spawning and syncing (automatic copying)

- **MultiplayerSpawner**: when the host spawns a player, Godot automatically creates the same player on every machine, including people who join later. We give it a *spawn function* so each copy gets the right name, look and position.
- **MultiplayerSynchronizer**: copies chosen variables from the authority to everyone, every frame. Ours syncs `net_pos, oxygen, carried_value, carried_count, facing, thrust, in_station, is_blacked_out, health, on_radar`. Other machines **smoothly glide** the player towards `net_pos` instead of snapping.

```
# world.gd: runs on EVERY machine with the same data, so all copies match
func _spawn_player(data: Variant) -> Node:
    var player: Player = PLAYER_SCENE.instantiate()
    player.name = str(data["id"])        # named after the peer id -> authority
    player.display_name = data["name"]
    player.look = data["look"]           # which of the 8 sprite looks
    player.position = station.spawn_point(player.look)
    return player
```

Idea 5: RPCs (calling a function on other machines)

An `@rpc` function can be called across the network with `.rpc()` (everyone) or `.rpc_id(peer)` (one machine; `rpc_id(1)` = the host). The annotation says who may call it:

Annotation	Meaning
`"any_peer"`	Anyone can call it (used for *requests to the host*).
`"authority"`	Only the node's authority can call it (used for *the host's decisions*).
`"call_local"`	Also run it on the machine that called it.
`"reliable"` / `"unreliable_ordered"`	Guaranteed delivery (events) vs. fast and droppable (positions 20× a second).

The pattern we use everywhere: request → host decides → broadcast

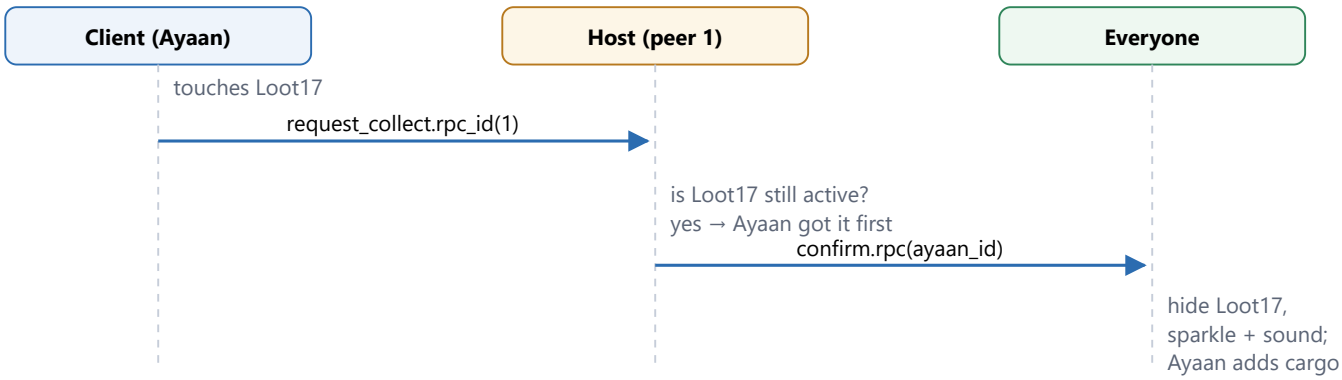


Figure 2: a loot pickup. If two players touch the same loot, the host confirms only the first request; the second finds it inactive and is ignored.

```gdscript
# loot.gd
@rpc("any_peer", "call_local", "reliable")
func request_collect() -> void:                # runs on the host
    if multiplayer.is_server() and active:
        confirm.rpc(multiplayer.get_remote_sender_id())

@rpc("authority", "call_local", "reliable")
func confirm(collector_id: int) -> void:       # runs on everyone
    ...set_collected()                           # hide it, respawn in 60 s
    if collector_id == multiplayer.get_unique_id():
        player.add_cargo(value)                  # only the winner gets the cargo
```

Buying works the same way: `GameState.request_buy()` → host's `buy_upgrade()` checks the price → host broadcasts the new totals with `sync_state.rpc(...)`, so every machine shows the same money.

Smart shortcut: the loot isn't networked at all

All 160 pieces of loot are created on every machine from the **same random seed (12345)** with the same names (`Loot0` … `Loot159`), so everyone already has an identical loot field. Only *pickups* travel over the network. Late joiners get a list of what's already been taken (`sync_loot`) plus the current money and upgrades (`send_state_to`).

Joining, step by step

1. Menu: type a name and code, click **Join** → `change_scene_to_file("world.tscn")`.
2. `world.gd` `_ready()` builds the world **first**, then calls `NetworkManager.join()`. (Building first means the spawner is ready before the host's messages arrive.)
3. On connecting, the client calls `register_player.rpc_id(1, name)`: "I'm here, spawn me".
4. The host sends the team state and taken loot, picks a free sprite look, and spawns the player, and the spawner copies that player to everyone.

> **Say this:** "Each player controls their own astronaut locally so it feels instant, but anything shared (money, loot, relics) goes through the host, which decides and broadcasts. The loot field is generated from the same seed on every machine, so we only send pickups, not 160 objects."

4 File-by-file tour

Read each card, then open the file and find the functions listed. Ordered from "start of the game" to "polish".

network_manager.gd `autoload`

Connecting players: host, join, leave, join codes, dev command-line flags.

- `host()` / `join(ip)` : create the ENet server or client (see section 3).
- `leave(error)` : disconnect, reset the team state, back to the menu (shows the error there).
- `local_ips()` + `_adapter_score()` : this laptop's real Wi-Fi/Ethernet IP first.
- `my_code()` / `code_to_ip()` : the 3-number join codes.
- `_parse_cmdline()` : dev flags like `--host`, `--join`, `--bot`, `--money=6000` to test quickly.

game_state.gd `autoload` `host-authoritative`

The team's shared money, upgrades and repair parts, plus all the balance tables and the keyboard controls.

- Balance tables at the top: `OXYGEN_COSTS`, `SPEED_COSTS`, `PART_COSTS`, `TETHER_COSTS`, `OXYGEN_CAPACITY`, `THRUST`, `MAX_SPEED`, `TETHER_LENGTHS`. The index is the current level.
- `add_money()`, `buy_upgrade()` : only act on the host; then `_broadcast()` → `sync_state.rpc(...)`.
- `sync_state()` : every machine stores the new values and emits `state_changed`; emits `game_won` once when parts reach 5.
- `announce(text)` : the event feed ("Ali sold 3 salvage for $73").
- `_register_inputs()` : WASD/arrows, E, R, 1–4 are added in code.

```
@rpc("any_peer", "call_local", "reliable")
func buy_upgrade(kind: String, buyer_name: String) -> void:
    if not multiplayer.is_server() or has_won(): return
    var cost := cost_of(kind)
    if cost < 0 or cost > money: return   # maxed out or can't afford
    money -= cost
    ...
    _broadcast()
```

> **A bug we found and fixed:** the host never saw the win screen. The host increased `parts` *before* the "did we just win?" check, so the check thought it had already won. Fixed with a `_celebrated` flag so `game_won` fires exactly once per round. Good story if judges ask about debugging.

menu.gd + menu.tscn

The main menu: name box, join-code box, Host / Join buttons (styled with our button images in `Imgs/`).

- `_build_ui()` : builds the whole menu in code (panel, text boxes, textured buttons).
- `_start(as_host)` : saves the name and turns the typed code into an IP with `code_to_ip()`, then loads `world.tscn`.

world.gd + world.tscn

The game world: starts networking, spawns players (host only), creates the loot and relics, music, background and darkness.

- `_ready()` : set up the spawner → `_spawn_loot()` → `_spawn_relics()` → music → background → darkness → then `host()` or `join()` .
- `register_player()` rpc : a client asks to be spawned; `_free_look()` picks a sprite nobody is wearing.
- `_spawn_loot()` : 160 loot at random angles/distances (450–3400 px) from seed 12345.
- `_on_peer_disconnected()` , `_on_join_timeout()` , `_on_server_disconnected()` : leaving and errors.

player.gd + player.tscn `the biggest file`

One astronaut. Its owner simulates it; everyone else sees a smoothed copy.

Function	What it does
`_enter_tree` , `_setup_replication`	Take authority (node name = peer id) and list the variables to sync.
`_ready`	Builds the sprite (one of 8 looks), collision capsule, helmet lamp, boot exhausts; turns on the camera for *your* player only.
`_physics_process`	Your player only: read input (or the bot) → move → rope physics → oxygen → E (sell / grab relic) → R (radar).
`_move_in_station`	Inside: gravity, walking, jetpack boots (platformer, like Terraria).
`_move_in_space`	Outside: zero-g. Input adds acceleration, there's a speed cap and slight drag, and you slow down when carrying cargo or a relic.
`_update_oxygen`	Air drains outside, refills inside. Empty tank → health drains 20/s (≈5 s) → blackout.
`_pull_ropes` , `rope_snapped` rpc	Tether physics and snapping (section 5).
`sell` , `add_cargo` , `toggle_grab`	Selling, picking up, grabbing/releasing relics.
`_black_out` , `_respawn`	Lose cargo, fade out, reappear at the station with full air.
`_process` , `_animate`	Every machine: glide remote players, pick the sprite frame (idle/run/fly/float/reach/dead), boot-jet exhaust, camera shake.
`fx` rpc	Effects everyone sees: coin fountain (sell), air puff (blackout), rope bits (snap).
`_draw`	Ropes, boot flames from both feet, name tag, carried value.

```
func _move_in_space(input: Vector2, delta: float) -> void:
    var load_factor := maxf(MIN_LOAD_FACTOR, 1.0 - CARRY_SLOWDOWN * carried_count)
    if carried_relic():
        load_factor = minf(load_factor, HeavyRelic.CARRIER_SPEED_FACTOR)
    velocity += input * GameState.thrust() * load_factor * delta      # accelerate (momentum!)
    velocity = velocity.limit_length(GameState.max_speed() * load_factor)
    velocity *= maxf(0.0, 1.0 - SPACE_DRAG * delta)               # slight drift slow-down
```

station.gd

The home base, drawn side-on: pressurised room, solid walls and ceiling, airlock doors, one-way floor, lights, beacon, escape pod.

- `_ready()`: the **Area2D** covering the room tells players when they're inside (`set_in_station`), plus solid walls with gaps for the airlocks and a **one-way deck** (you can fly up through it from below).
- `make_light()`: builds a soft round `PointLight2D`; also used for helmet lamps.
- `_launch_pod()`: the win finale. The engine fires, the pod rumbles for 0.8 s, then a tween lifts it 1600 px with an exhaust trail and a glow.
- `_draw()`: hull, portholes, solar panels, SHOP/RADAR holograms, pod with 5 lights (green per part fitted).

loot.gd + loot.tscn

One floating piece of salvage. Value rises with distance: `10 + (distance/100)^1.35`.

- Tiers by distance: **scrap** (<1400 px), **gold bar** (<2400), **relic** (beyond).
- `request_collect` / `confirm`: the pickup handshake (Figure 2). `set_collected` hides it; it respawns after 60 s.
- Bobs and wobbles with `sin()` in `_process`; shows a floating "+$32" popup.

heavy_relic.gd `host-simulated`

Three $500 relics that need two carriers. The host runs them and streams their state 20× a second.

- `request_grab` / `request_release` rpc: ask the host to add or remove you as a carrier (must be within 90 px).
- `_simulate` (host): drop carriers who left, blacked out or drifted >220 px away ("lost their grip"); with 2+ carriers, move the relic towards the **midpoint** of the carriers; if it reaches the station, `_deliver()`.
- `sync_state` rpc, unreliable: position + carriers to everyone; clients glide to it.
- `_deliver`: +$500, announcement, cyan burst + sound, respawns 90 s later.

```
if carriers.size() >= CARRIERS_NEEDED:
    var center := Vector2.ZERO
    for id in carriers:
        center += _player(id).global_position
    center /= carriers.size()                          # midpoint of everyone holding on
    position = position.move_toward(center, speed * delta)
```

`tethers.gd` helper

Works out who is roped to whom. No networking needed: every machine computes the same answer from synced data.

```
static func links(tree: SceneTree) -> Array:
    # divers = everyone NOT on radar duty, sorted by sprite look (same order everywhere)
    if divers.size() == 1:            # a lone diver is roped to the station
        result.append([divers[0], null])
    for i in range(0, divers.size() - 1, 2):
        result.append([divers[i], divers[i + 1]])      # pairs: 1+2, 3+4, ...
    if divers.size() % 2 == 1:
        result.append([divers[-2], divers[-1]])        # odd one out makes a chain of 3
```

`hud.gd` CanvasLayer

Everything on screen that isn't the world: built entirely in code.

- Top left: team money, **O2 bar**, **HP bar**, cargo, upgrade levels, distance from station.
- Shop panel (in the station) and **radar** (only on radar duty, R): `_draw_radar()` draws loot dots, relics, players with their oxygen %, and rope lines, by scaling world positions down with `_radar_point()`.
- `_draw_arrow()`: gold arrow pointing home, size from 1.0 (close) to 0.35 (3000 px away).
- `_hint()`: the context line at the bottom ("Press E to sell", "TETHER STRAINING...").
- Red **vignette shader** pulsing when suffocating or about to snap; blackout fade; win screen with confetti after the pod launch; join code in the bottom right for the host.

```
func _radar_point(world_pos: Vector2, center: Vector2, radius: float) -> Vector2:
    var offset := world_pos / RADAR_RANGE * radius     # 3600 px of world -> radar edge
    return center + offset.limit_length(radius - 4.0)  # far things stick to the rim
```

`space_background.gd` + `nebula.gdshader`

The animated pixel-art nebula behind the game, which turns from calm blue to danger red the further you are from the station.

- A **shader** is a tiny program that runs on the graphics card and colours every pixel every frame.
- The nebula shader is adapted from **Deep-Fold's Pixel Space Background Generator** (MIT licence): fractal noise makes cloud shapes, `floor(uv * pixels) / pixels` makes the chunky pixels.
- Our change: two colour palettes mixed by a `danger` value (0 near home, 1 at 300 m) that `space_background.gd` sets every frame from *your* distance; it also scrolls with the camera for a sense of depth. Stars and a planet are drawn with parallax.

```
// nebula.gdshader
vec3 calm  = texture(colors_calm,   vec2(col_value, 0.0)).rgb;
vec3 scary = texture(colors_danger, vec2(col_value, 0.0)).rgb;
vec3 col = mix(calm, scary, danger);      // 0 = all blue, 1 = all red
```

`effects.gd` `helper`

Particle effects.

- `Effects.burst(...)` : one-off puff of glowing pixels that deletes itself (pickups, selling, blackout, snap, relic).
- `Effects.thruster(...)` : looping exhaust (both boots, the escape pod).
- Particles are **unshaded + additive**, so they glow even in the darkness.

`sounds.gd` `helper`

Sound effects and music.

- `Sounds.play(self, "pickup", vol, pos)` : one-off sound; with a position it's **positional** (quieter far away).
- `Sounds.loop("music")` : looping player (music, your boot jets).
- 12 files in `sounds/` : Kenney.nl (CC0) + music by yd (CC0).

`dev_bot.gd` `testing tool`

An autopilot (`--bot` flag) that collects the nearest loot, comes home when full or low on air, sells, buys upgrades and keeps its rope slack. We used it to test multiplayer with two bots and no humans, including headless runs with no window.

> **How we tested:** we ran a host and a client on one laptop (Debug › Customize Run Instances, or headless with `--host --bot` / `--join --bot`) and checked the logs: joins, pickups, sales, purchases, rope snaps, suffocation timing (exactly 5 s) and the win. We also took automatic screenshots to check the visuals.

5 Mechanics deep dives

Movement: two physics modes

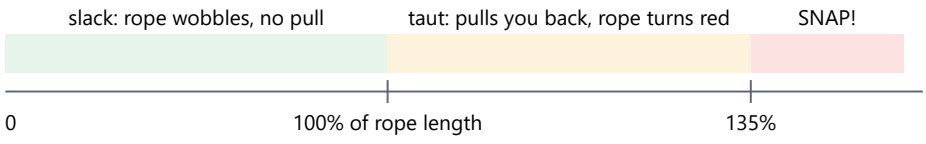
Inside the station (`_move_in_station`)
Gravity 900 px/s^2, walk 230 px/s, jetpack boots push up. It feels like a platformer. The station's Area2D flips `in_station` as you cross the airlock.

Outside (`_move_in_space`)
Zero-g: keys *accelerate*, you keep drifting (momentum), with a speed cap that upgrades raise (300 → 520 px/s) and slight drag. Cargo slows you, down to half speed.

Oxygen and health

Tank sizes 25 / 38 / 55 / 75 seconds (upgrade levels). Air refills 30× faster than it drains once you are inside. At 0 air you don't die instantly: **health drains 20 per second (100 → 0 in 5 s)**, the screen edges pulse red and an alarm beeps. Blackout = you lose your cargo, fade out, and respawn at the station 1.6 s later.

Tethers (rope physics)



Rope length: 50 / 70 / 95 / 125 m (Longer Tether upgrade, key 4). A lone diver's station rope is 3× longer.

Figure 3: what happens as two roped players drift apart.

- **Who's roped:** `Tethers.links()` : automatic pairs; radar duty (**R**) unclips you and re-pairs the others.
- **The pull:** each player's own machine adds a spring force towards their partner when the rope is longer than its length: `min(stretch × 6, 320)` . It's capped *below* jetpack thrust, so if you keep pushing you **can** snap it, after about 3 s of red-rope warning.
- **Snap:** past 135% → `_black_out("…tether snapped")` and `rope_snapped.rpc_id(partner)` , so **both** players go down together.
- **Grace for new ropes:** a new rope starts as long as the current gap and reels in at 90 px/s, so being paired with someone far away doesn't kill you instantly.
- If one player is towed rather than resisting, they get dragged along. It only snaps when you pull against each other.

Darkness and the radar

A `CanvasModulate` darkens the whole world to 7% brightness. Each player has a `PointLight2D` helmet lamp (240 px radius) and the station has a big warm light, so divers only see what's near them. The radar (HUD) isn't affected by the darkness, so the radar operator sees everything: loot by tier colour, relics as cyan diamonds, teammates with their oxygen %, and the ropes. One constant, `DARKNESS_ENABLED` , turns the darkness off.

Economy and winning

Everything goes to one shared team wallet. Total cost of everything: oxygen 850 + jetpack 850 + tether 850 + parts 2,800 = **$5,350**. Parts cost 250 / 400 / 550 / 700 / 900. The 5th part triggers `game_won` → camera zooms out → pod launch (3.5 s) → win screen with confetti and jingle → host can press "Play again" (`GameState.restart`).

6 Judge Q&A drill

Cover the answers and say yours out loud. Short, confident answers win.

How does it fit the Discord Games theme?

> Communication is required: divers are blind, so the radar operator must guide them; divers are roped together, so pairs must agree on a direction or the rope snaps; relics need two people pulling the same way.

How does the multiplayer work?

> Godot's high-level multiplayer over ENet. One player hosts (server, peer id 1) on UDP port 9999; others join with a 3-number code that's turned back into the host's IP. A MultiplayerSpawner copies players to every machine and a MultiplayerSynchronizer streams each player's position and stats.

What stops two players picking up the same loot, or double-spending money?

> The host decides. Clients send requests (`request_collect`, `buy_upgrade`); the host checks and broadcasts the result, so the first request wins and money can't go negative.

Why is movement client-side? Can't players cheat?

> Client-side movement feels instant with no lag. It's a co-op game among friends, so cheating isn't a concern, but anything shared (money, pickups, relics) is still decided by the host.

How do you sync 160 pieces of loot?

> We don't: every machine generates the same field from the same seed (12345) with the same node names. Only pickups go over the network, and late joiners get a list of what's already gone.

What happens if someone joins mid-game?

> The spawner automatically sends them every existing player; the host sends the money/upgrades (`send_state_to`) and taken loot (`sync_loot`); relic positions stream continuously anyway.

How do the tethers work?

> Everyone not on radar is paired automatically. Each machine applies a capped spring force to its own player when the rope is stretched past its length; at 135% it snaps and both black out. The pairing is computed identically on every machine, so it needs no extra networking.

How does the radar know where everything is?

> Every machine already has every player (synced) and every loot/relic, found via groups. The HUD scales their world positions down to a 320-pixel circle (3,600 px of world = the edge).

How is the darkness done?

> A CanvasModulate darkens the world layer; helmet lamps and the station are PointLight2Ds. The HUD and background are on separate canvas layers, so they stay bright.

What was the hardest bug?

> Two good ones. (1) The host never saw the win screen, because it bumped the part count before the win check; fixed with a once-per-round flag. (2) Join codes sometimes used a VirtualBox network adapter; fixed by ranking adapters so real Wi-Fi comes first.

How did you test multiplayer without lots of people?

> We wrote an autopilot bot (`--bot`) and ran host + client instances on one laptop, including headless with no window, then checked the logs and took automatic screenshots.

Which parts did you make vs. use?

> All the gameplay code is ours. Assets: Spacers sprites by zwonky (CC0), sounds from Kenney.nl (CC0), music by yd (CC0), and the nebula shader adapted from Deep-Fold's generator (MIT). All are credited in the README.

Can you play over the internet?

> Right now it's LAN (same Wi-Fi). For internet play we'd add port forwarding, a relay server, or Steam networking. That's our next step.

What would you add with more time?

> Meteor hazards the radar operator warns about, internet play, proper voice-chat integration, more relic types, and balancing from playtests.

Why Godot?

> It's free and open source, fast for 2D, and has high-level multiplayer built in (spawning, syncing and RPCs), which let us build an 8-player networked game within a hackathon.

Why build the UI in code instead of the editor?

> Scene files merge badly in git when several people edit them. Building in code kept scenes tiny, so the whole team could work in parallel without conflicts.

7 Cheat sheet

Controls

Key	Action
WASD / arrows	Walk + jet boots (inside), thrust (outside)
E	Sell (inside) / grab or release relic (outside)
R	Take or leave the radar (inside; unclips your rope)
1 / 2 / 3 / 4	Buy oxygen / jetpack / pod part / longer tether
Esc	Leave to menu

Networking facts

Library	ENet (Godot high-level multiplayer)
Port	UDP 9999
Max players	8
Host peer id	1
Join code	last 1–2 numbers of host IP
Join timeout	10 s
Relic sync rate	20× per second
Loot seed	12345 (160 pieces)

Demo-day tips

- Same Wi-Fi for all; if it fails, use a **phone hotspot**.
- Host must **Allow** Godot in the Windows Firewall (tick Public too).
- Quick win demo: host with launch argument `--money=6000`, buy everything with 1–4.
- Show: radar call-outs, a relic haul, a rope snap, the pod launch.

Key numbers

What	Value	File
Oxygen tank	25 / 38 / 55 / 75 s	game_state
Max speed	300 → 520 px/s	game_state
Rope length	50 / 70 / 95 / 125 m	game_state
Upgrade costs	$100 / 250 / 500	game_state
Pod parts	$250 / 400 / 550 / 700 / 900	game_state
Suffocation	5 s (20 HP/s)	player
Rope snaps at	135% of length	tethers
Rope pull cap	320 (below thrust)	player
Relic	$500, 2 carriers, 90 s respawn	heavy_relic
Loot respawn	60 s	loot
Loot value	10 + (dist/100)^1.35	loot
Helmet lamp	240 px radius	player
Radar range	3600 px (360 m)	hud
Units	10 px = 1 m	hud
Pod launch	3.5 s before win screen	station

Credits (all free to use)

- Sprites: *Spacers* by zwonky (CC0)
- Sound effects & jingles: Kenney.nl (CC0)
- Music: *Observing The Star* by yd (CC0)
- Nebula shader: adapted from Deep-Fold's Pixel Space Background Generator (MIT)

> **The three sentences to remember:** (1) "Communication is the mechanic: blind divers, a radar operator, ropes and two-person relics." (2) "Each player runs their own movement for instant feel; the host decides anything shared, like money, pickups and relics." (3) "The loot field comes from a shared seed, so we only network pickups, and we tested everything with our own autopilot bots."